11 - Stratégie de tests

Ce document ne remplace pas le [document de tests et de validation](02-document-tests-validation.md) (cas détaillés, registre d'anomalies, PV de recette) : il explique **comment** la vérification est construite, **pourquoi** à ce niveau-là, et **ce qu'elle ne prouve pas**. C'est le document qu'on lit quand on se demande « vos tests, ils servent à quoi ? ».

1. Le principe : la pyramide, appliquée pour de vrai

La pyramide des tests est souvent citée puis contredite par les chiffres. Voici les nôtres, mesurés sur la révision courante (`node scripts/run-all-tests.mjs`).

Éta	Nature	Suite	Volume	Durée
0	Analyse statique & audits documentaires	`scripts/strict_types_lint.php` (`declare_strict_types`, types natifs complets, zéro ternaire) + `tests/security/controles.sh` (10 contrôles + autotest) + `scripts/verifier_fiches_uc.py` + `tests/python/test_audit_structurel.py` + autotest `scripts/wasm_verdict.mjs`	**158 fichiers** PHP lintés, 0 violation - **10/10** contrôles sécurité - **46** contrôles fiches manuelles (`164` scénarios + `36` couples)	< 2 s
1	Unitaires purs	`tests/php/unit_tests.php` + `tests/phpunit/**` (9 classes : `Text`, `Filter`, `Prix`, `Triggers`, `JsonStore`, `SecurityContract`, `AlternativeImage`, `JournalEtTable`, `ConservationEtFuseau`)	suite PHP unitaire + **50** tests PHPUnit 11	~ 6 s
2	Contrats statiques & métier ciblés	`contract_http.php`, `css_contract.php`, `responsive_contract.php` + 31 suites spécialisées `tests/php/*_test.php` (confirmations 2 étapes, historique annulation, doublons catégories, défaut stock `DELTA`, mentions LCEN, heure UTC/DST, rétention 24 mois, nettoyage, visuels, alignements...)	**42 suites PHP** exécutées sous PHP 8.2 et PHP 8.5 sans dépréciation	~ 15 s
3	Règles de gestion sur le moteur	`tests/php/run_tests.php` (`T-01` -> `T-29` du CDC §6), `tests/php/engine_smoke.php`, `tests/php/json_concurrency_stress.php`	**29/29** scénarios + contrôles moteur + stress concurrent JSON	~ 10 s
4	Fonctionnalités via dépôts & fixtures isolées	`tests/php/features_test.php` (UC-01 -> UC-14), `catalogue_stock_test.php`, `nouveautes_test.php` (magasins temporaires dédiés, `data/minishop` jamais lu ni modifié)	**7 656+** cas catalogue + parcours UC complets	~ 8 s

5	Intégration SQL réelle & migrations	`tests/php/sql_integration.php` (`tests/sql/run_tests.sh` + étape 6 PDO : `sp_cancel_order`, `ANNULATION_VIA_PARCOURS_DE` `EXPEDIEE` sans retour stock, UTC `+00:00`, `revenueReport`, `RetentionComptes`) + `tests/ph /migration_mariadb_test.php`	**29** scénarios SQL + étape 6 + migrations sur **MySQL 8.0** et **MariaDB 11.8**	~ 30 s
6	Parcours HTTP E2E, DOM bilingue FR<->EN & CLI Python	`scripts/wasm-e2e.mjs` + 23 suites `scripts/ui-e2e-*.mjs` + `tests/node/preferences_test.mjs` + `tests/node/e2e_isolation.mjs` + `tests/python/test_{retention_cli,nettoyage_cli,archive_restreinte,preparer_sql_lineserv,auth_403_http}.py`	**338** contrôles HTTP E2E isolés + **48 vues** `FR -> EN -> FR` & **44 erreurs** métier + AJAX `panier.js`	~ 90 s
6 b	Navigateur réel headless (CI)	`tests/node/responsive_navigateur.mjs` (Playwright Chromium : 320, 375 et 1440 px, français et anglais, avec et sans JavaScript)	cartes `data-label`, pages légales FR/EN, panier AJAX EN, annulation admin `EXPEDIEE` sar JS	~ 45 s
7	Performance & volumétrie	`scripts/capture_perf.sh` -> `gen_volumes.sh` + `tests/perf/mesurer_sql.sh`	5 requêtes types + `EXPLAIN` + p95 de la recherche	~ 60 s
8	Suites de recette manuelle humaine	[`docs/13-fiches-recette-uc.md`](13-fiches-recette-uc.md) + [`docs/13-suivi-recette-uc.csv`](13-suivi-recette-uc.csv) + [`docs/14-guide-lineserv.md`](14-guide-lineserv.md)	**164 scénarios** (`UC-01..14` + `TCX-01..24`, hors responsive multi-largeurs) + **36 couples** = **200 lignes `NON JOUÉ`**	campagne humaine

Total : les volumes détaillés figurent ci-dessus ; les scénarios catalogue varient avec le nombre de produits. La matrice catalogue MariaDB reste à jouer.

Pourquoi cette forme, et pas 400 tests unitaires avec des doublures partout : l'essentiel du sujet est une **base de données** (8 entités, 17 procédures, 16 déclencheurs, 1 fonction, 3 vues). Un test unitaire avec un `PDO` simulé prouverait que le PHP appelle bien une chaîne de caractères - c'est-à-dire rien. Les règles de gestion (stock, immutabilité du prix, transitions de statut, total recalculé) sont donc vérifiées **là où elles vivent réellement** : étage 3 sur le moteur, étage 5 sur MariaDB. La pyramide est respectée dans son esprit (beaucoup de tests rapides en bas) mais son sommet est anormalement lourd, et c'est assumé : c'est le sujet.

2. Les trois natures de test, et ce qu'elles distinguent

2.1 Tests de règles (étage 3) - le moteur, sans HTTP

`tests/php/run_tests.php` exécute les 29 scénarios T-01 -> T-29 du cahier des charges sur `app/Model/Data/JsonStore.php`, qui réimplémente contraintes, déclencheurs et transactions. L'intérêt n'est pas de « tester du JSON » mais de **vérifier la règle indépendamment de l'application**

: aucun contrôleur n'est instancié, aucune session n'existe. Si quelqu'un contourne le formulaire, la règle tient encore.

`tests/php/engine_smoke.php` (44 contrôles) vérifie les **hypothèses de cette réimplémentation** : que les deux pilotes (`JSON` et `SQL`) exposent la même API, que `SqlStore` n'ouvre pas de connexion sans PDO, qu'il n'assemble aucune requête par concaténation, que `LIKE` ignore casse et accents. Sans cet étage, un test de règle vert sur JSON pourrait passer sur un moteur qui ne se comporte pas pareil.

2.2 Tests d'intégration SQL (étage 5) - la seule preuve que le SQL fonctionne

`tests/php/sql_integration.php` est une **enveloppe mince** autour de `tests/sql/run_tests.sh` (qui existait déjà) : il crée une base dédiée `minishop_test`, y charge `sql/01` -> `sql/04`, teste également le repository sur `SqlStore` et la procédure de recherche, joue les 29 scénarios, puis **supprime la base**.

Depuis l'audit global, cette enveloppe refuse par défaut une base existante et refuse toujours le nom `DB_NAME` configuré. `--reinitialiser-base-existante` autorise explicitement la suppression : serveur de test jetable uniquement. La protection du nom ne détecte pas les bases utilisées par d'autres applications. Le harnais shell appelé directement ne bénéficie pas de ces garde-fous.

Le mot « mince » est important : réécrire les 29 cas en PHP aurait produit deux jeux de tests qui divergent au premier changement de schéma. Le harnais shell est la référence ; le PHP n'ajoute que l'isolation.

L'isolation est le point délicat : `tests/sql/fixture.sql` réinitialise l'état **avant chaque scénario** - lancée sur la base de développement, la campagne effacerait les commandes de démonstration. L'enveloppe refuse le nom de base applicative configuré ; cela ne protège pas les bases des autres applications. `--garder` permet d'inspecter le résultat.

Un détail découvert sur la machine de recette : le compte applicatif `minishop` n'a volontairement de droits que sur `minishop.*` (moindre privilège), donc `CREATE DATABASE minishop_test` est refusé - `1044 Access denied`. Ce n'est pas un bug du test, c'est la politique de droits qui fonctionne. La base annexe s'ouvre donc **explicitement**, par `php scripts/db_init.php --test-db` : elle crée `minishop_test` et y accorde les droits, **sans** toucher au mot de passe du compte ni élargir quoi que ce soit sur la base de production. Le test, lui, ne demande jamais `sudo` de lui-même. Puisque cette commande crée une base existante, le test la refuse ensuite par défaut ; sa réinitialisation exige l'option destructive explicitée ci-dessus, après vérification et uniquement sur serveur jetable.

Dégradation : sans client `mariadb`/`mysql` dans le `PATH`, ou sans serveur joignable, le test annonce **NON JOUÉ** et sort en code 2. `scripts/run-all-tests.mjs` distingue ce cas d'un échec - un test qu'on ne peut pas jouer doit le dire, pas mentir en vert ni échouer pour de mauvaises raisons.

2.3 Tests de bout en bout (étage 6) - ce que l'utilisateur voit

`scripts/e2e.mjs` exécute le parcours WASM : les requêtes passent par le front controller dans le runtime PHP-WASM, sans navigateur ni serveur HTTP réel. Il vérifie les codes HTTP, le contenu des pages, les jetons CSRF et la persistance dans le magasin JSON. 268 vérifications couvrant inscription, connexion, recherche, panier, commande, suivi, administration.

Depuis l'audit global, **le test ne charge pas les données de l'hôte** : il initialise et modifie uniquement un système de fichiers PHP-WASM jetable, sans rapatriement ni restauration. Le test `tests/node/e2e_isolation.mjs` vérifie en copie temporaire que les fichiers hôte factices restent inchangés.

3. Les tests de contrat - la partie qui attrape ce que les autres laissent passer

Trois suites tournent **sans exécuter l'application**. Elles sont rapides, et elles attrapent une catégorie de défaut que ni les tests unitaires ni l'E2E ne voient : la **cohérence entre deux artefacts écrits séparément**.

Suite	Ce qu'elle garantit	Exemple de défaut attrapé
`contract_http.php` (27)	Chaque lien et chaque `action=` d'une vue pointe vers une route **déclarée** ; toute route `/admin/*` porte une garde ; aucune écriture en `GET` ; les en-têtes de sécurité sont émis	un `<a href="/commande/suivi">` après renommage de la route ; une route d'administration sans `exigerAdmin()`
`css_contract.php` (69)	Les jetons du design system sont réellement définis et utilisés ; les contrastes calculés passent AA ; la CSP n'autorise pas d'origine externe	une couleur écrite en dur qui contourne le jeton ; un `style=` inline bloqué par la CSP
`responsive_contract.php` (50)	La mise en page tient de 320 px à 1536 px (WCAG 1.4.10) : pas de largeur fixe, tableaux en conteneur défilant, cibles tactiles >= 24 px	une grille `min-width: 900px` qui provoque un défilement horizontal sur mobile

Ces tests sont **statiques** : ils analysent les fichiers. C'est une limite réelle (aucune validation de navigateur dans ces trois scripts), compensée en CI par `tests/node/responsive_navigateur.mjs` sous Chromium headless (320, 375 et 1440 px) - l'équipe ayant choisi de retirer les campagnes responsive multi-largeurs des suites manuelles (trop longues à dérouler à la main).

4. Ce qui n'est pas joué en sandbox, et pourquoi c'est écrit

La sandbox de développement n'a ni PHP natif, ni MySQL, ni navigateur. Elle exécute les suites PHP via un runtime WASM (`scripts/wasm_run.mjs`, `@php-wasm/node`), ce qui suffit pour les étages 0 à 4 et 6.

Suite	Sandbox	Machine avec MariaDB	Conséquence
Étages 0-4, 6 (PHP + E2E)	[x] joués	[x] joués	-

`sql_integration.php` (étage 5)	NON JOUÉ (`exec` indisponible)	[x] joué	joué sur la machine de l'équipe, résultat consigné
`coverage.php`	NON JOUÉ (lance des processus enfants)	[x] joué	voir §6
`capture_perf.sh` (étage 7)	NON JOUÉ (pas de serveur)	[x] joué	mesures consignées dans `tests/perf/dernieres_mesures.txt`
Navigateur réel, lecteur d'écran	NON JOUÉ	partiellement	vérifications manuelles, §5 du doc de tests

Règle tenue : un test non jouable annonce « NON JOUÉ » et sort en code 2 ; il n'est jamais présenté comme vert. Une case cochée sans résultat derrière est un mensonge qui se paie en soutenance.

5. La couverture, mesurée honnêtement

`scripts/coverage.php` ne mesure **pas** la couverture de lignes : elle exigerait Xdebug ou PCOV, c'est-à-dire une dépendance à installer pour noter le projet (contraire à `ENF-20`). Il mesure ce qui est vérifiable sans rien installer : **quels fichiers de `app/` sont chargés par au moins une suite**.

Un fichier jamais chargé n'est pas « mal couvert », il n'est **jamais exécuté** : aucune suite ne peut détecter sa régression. C'est le bon signal pour repérer du code mort.

```
php scripts/coverage.php              # tableau par suite + pourcentage
php scripts/coverage.php --liste      # fichiers jamais chargés
php scripts/coverage.php --min=80     # échoue sous 80 % (CI)
```

Ce que cette mesure ne dit pas, et qu'il faut dire à sa place : un fichier chargé n'est pas un fichier correctement exercé ; les vues (`app/View`) ne sont chargées qu'à l'affichage, donc leur couverture par les suites PHP est structurellement faible - c'est l'E2E qui les exerce, et il n'entre pas dans ce calcul.

6. Le harnais se vérifie lui-même

Un dispositif de test qui ne peut pas échouer ne prouve rien. C'est arrivé, et c'est écrit ici parce que le défaut était invisible.

En mode PHP-WASM, `scripts/wasm_run.mjs` propageait `result.exitCode`. Cette valeur ne vient pas du CLI mais du gestionnaire de **requêtes** PHP. Mesures sur ce dépôt :

Script	`exitCode` renvoyé
terminé normalement	**1**
`exit(0)` / `exit(1)` / `exit(3)`	**0**
exception non attrapée (erreur fatale)	**0**

Autrement dit : dans la sandbox, `run-all-tests.mjs` comptait les échecs comme des succès. Le verdict est maintenant calculé sur des signaux observables (`scripts/wasm_verdict.mjs`) :

- erreur fatale ou de syntaxe PHP -> 255 ;
- au moins un marqueur « » dans la sortie -> 1 (les suites de ce dépôt n'emploient ce marqueur que sur un contrôle en échec) ;
- « NON JOUÉ » et aucun « [x] » -> 2 ;
- sinon -> 0.

Ce module porte son **autotest de 7 cas**, branché en tête de `run-all-tests.mjs`. Sans lui, une régression du verdict rendrait à nouveau toutes les suites vertes par construction - et c'est exactement le genre de panne qui ne se voit jamais, puisqu'elle produit un résultat rassurant.

Vérification de bout en bout : une suite volontairement sabotée fait sortir `run-all-tests.mjs` en **1** ; restaurée, en **0**.

7. Comment on lance tout

```
# tout, dans l'ordre, arrêt au premier échec
node scripts/run-all-tests.mjs        # sandbox (PHP-WASM)
php scripts/run-all-tests.php         # machine avec PHP natif

# une suite
node scripts/wasm_run.mjs tests/php/run_tests.php
php tests/php/run_tests.php

# SQL et performance : uniquement sur un serveur de test jetable.
# L'intégration refuse une base déjà existante par défaut ; voir la section 2.2.
# Les outils SQL et de performance directs peuvent réinitialiser leurs cibles.
```

8. Auto-évaluation

Ce qui tient. Les 29 règles de gestion sont vérifiées deux fois - sur le moteur et sur MariaDB - et 15 des 29 scénarios SQL contournent volontairement l'application pour prouver que la règle ne dépend pas du code PHP. Les parcours utilisateur sont exercés en HTTP réel, pas en appel de méthode. Les tests de contrat couvrent une zone (liens, CSP, responsive) que la pyramide classique laisse vide.

Ce qui ne tient pas, et qu'on assume.

1. **Pas de couverture de lignes.** Assumé pour ne pas ajouter de dépendance ; le substitut (fichiers chargés) est plus faible et le dit.
2. **Les tests de contrat sont statiques.** Une expression régulière sur un gabarit n'est pas un navigateur : un lien construit dynamiquement échappe au contrôle (la suite l'annonce, elle ne le juge pas).
3. **Pas d'accessibilité automatisée.** Contrastes et cibles tactiles sont calculés ; la navigation au lecteur d'écran reste manuelle.
4. **Le harnais de tests a menti pendant un temps.** Les campagnes « tout est vert » conduites en sandbox avant la correction du §6 ne prouvaient rien sur les codes de sortie. Les

suites ont ensuite été rejouées, et le harnais se vérifie désormais lui-même à chaque exécution. Le point est signalé ici plutôt que corrigé en silence : un rapport de tests qui a été faux une fois doit le dire.

5. **Pas de test de charge HTTP.** `C-01` simule 20 réservations concurrentes sur la base ; l'application sous charge réelle n'est pas mesurée.

6. **La campagne SQL dépend de l'environnement.** Elle n'est pas exécutable en sandbox ; elle a été jouée sur la machine de l'équipe, et le rapport `tests/sql/rapport_tests_sql.md` en est la trace.

Un défaut trouvé par ces tests et **non corrigé** est documenté plutôt que masqué : l'agrégation `sp_revenue_report` reste en `ALL` (anomalie A-08, 17 ms mesurées, sous le seuil `ENF-01`) - corriger aurait ajouté un index de couverture non justifié par la volumétrie retenue.

Information juridique, mentions légales LCEN et protection des données

`tests/php/information_juridique_test.php` et `tests/php/mentions_legales_test.php` vérifient les contrats statiques de navigation, contenu, cookies/stockage local (`sessionStorage` et `document.cookie` interdits dans `public/js/*.js`), mentions légales étudiantes conformes à l'article 6, III, 2 de la LCEN (Point 20 : co-éditeurs Johan Polsinelli et Chahd Gouiaa, hébergeur IUT `linserv-info-01.campus.unice.fr`), information précontractuelle et design ; ces contrôles ne constituent pas un avis juridique. Le parcours `scripts/wasm-e2e.mjs` (338 vérifications, sections T1 à T15) et `scripts/ui-e2e-mentions.mjs` testent les 12 pages d'information/légales, `/preferences` sans compte ni traceur, l'encadré de garanties rendu, l'avertissement de rétractation, le cookie de session observé et le contact (réussite, message de plus de 500 caractères, erreur d'écriture, CSRF et XSS). Les données de test restent isolées dans un magasin temporaire dédié. Détails et réserves dans `docs/12-information-juridique.md`.

Dates UTC, fuseau horaire local et traduction intégrale FR <-> EN

- `tests/php/date_ajout_test.php`, `tests/php/heure_test.php` et `tests/phpunit/ConservationEtFuseauTest.php` vérifient le stockage UTC de bout en bout (`SET time_zone = '+00:00'` sur PDO), les conversions `Europe/Paris` et fuseaux IANA (dont jours de bascule DST de 23 h et 25 h), ainsi que les bornes UTC des filtres `/admin/commandes` et du tableau de bord `/admin`.

- `tests/node/preferences_test.mjs` exerce dans JSDOM la page `/preferences` (choix manuel, géolocalisation locale CC0 `tz-lookup.js` sans requête réseau externe ni coordonnée stockée, `localStorage` indisponible, réinitialisation), l'aller-retour complet `FR -> EN -> FR` sans perte sur les **48 vues** de l'application, la traduction des **44 erreurs métier** (`BusinessError`) et le comportement anglais de `panier.js` (`Adding...`, `Cart, 1 item` / `2 items`).

- `tests/node/responsive_navigateur.mjs` valide dans un vrai Chromium headless (en CI, job `responsive-browser`) les largeurs **320, 375 et 1440 px** en français et en anglais, avec et sans

JavaScript : absence de débordement horizontal, cartes mobiles étiquetées `data-label` sur les tableaux d'administration, pages légales EN, annonce AJAX du panier EN et confirmation d'annulation admin sans JS d'une commande `EXPEDIEE`.

Dépréciations PHP et isolation des tests

• `python3 scripts/verifier_deprecations.py` lance `php -l` sur les fichiers PHP du code applicatif, des scripts et des suites, avec toutes les erreurs affichées. Il relève aussi un `Deprecated` même si PHP rend le code 0. Syntaxe et dépréciations sont vérifiées avec **le PHP CLI réellement lancé**, pas avec une version théorique. Il n'exécute pas les fichiers ni les dépendances externes.

• Les runners des suites PHP chargent `tests/php/deprecations_bootstrap.php` : `E_DEPRECATED` et `E_USER_DEPRECATED` sont signalés même sous `@`, et un marqueur fait échouer la suite. Cela ne couvre que le code réellement exécuté et peut être contourné si ce code remplace le gestionnaire d'erreur. Les parcours E2E HTTP et les fichiers exécutés directement hors runner n'y sont pas inclus.

• Pour éviter la base et les données E2E, utiliser `php scripts/run-all-tests.php --sans-sql --sans-e2e`. Python 3 et PHP CLI 8.2+ avec `mbstring` sont requis ; ni npm ni MariaDB ne sont requis pour ce mode. Sans options, SQL crée puis supprime une base de test et refuse les bases déjà existantes par défaut ; serveur isolé obligatoire. L'E2E ne modifie plus `data/` sur l'hôte.

• En l'absence de PHP CLI natif, le runner Node/WASM annonce l'analyse de compilation NON JOUÉE. Le garde d'exécution y utilise PHP-WASM 8.2 ; cela ne démontre rien sur les dépréciations apparues dans PHP 8.4/8.5. Rejouer avec le PHP de l'hébergeur et celui du serveur web ; les tests n'analysent ni JavaScript/CSS ni bibliothèques externes.

• Les autotests couvrent notamment un faux compilateur qui renvoie 0 malgré un `Deprecated`, une erreur de syntaxe, PHP absent, un dossier vide, les liens symboliques, une dépréciation utilisateur masquée et la réécriture du schéma de démo dans une **copie temporaire**. Aucun serveur SQL n'est contacté par ces autotests.

Code potentiellement inutilisé et inventaire applicatif

`scripts/code_mort.php` inspecte les tokens des PHP de `app/` et `public/` ; il cherche des références dans ces dossiers ainsi que dans les scripts et tests PHP. Il signale les paramètres non cités, les variables locales écrites une seule fois et les fonctions/méthodes sans autre mention textuelle. Ces signaux sont **à examiner**, pas des preuves de code mort. Les appels dynamiques, l'héritage, la réflexion, les API externes et certains modes de lecture indirecte rendent l'analyse nécessairement incomplète ; JS/CSS/SQL et variables globales ne sont pas analysés de cette façon.

La commande normale compare les candidats au référentiel `scripts/code_mort_reference.json` : un nouvel indice ou un indice disparu échoue jusqu'à examen et mise à jour du référentiel. `--rapport` affiche les candidats connus et `--json` les exporte. Les tests couvrent des exemples positifs, négatifs, appels de callback, propriété promue, référence à une propriété statique,

lectures indirectes, ainsi qu'un scénario CLI natif dans une copie temporaire. L'inventaire `docs/16-audit-code-mort.md` documente chaque fichier versionné de `app/` et `public/` ; son test vérifie la liste et les fichiers de zéro octet, pas que le rôle décrit reste correct après toute modification.

Page 403 et rôles administrateurs

Un administrateur connecté sans le rôle SUPER reçoit désormais un statut HTTP 403 et la page « Accès refusé » ; un visiteur non connecté est toujours redirigé vers `/admin/connexion`. Les routes GET et POST sont protégées avant l'action, y compris avec un jeton CSRF valide. Les parcours E2E WASM incluent l'absence de mutation et l'accès SUPER. La CI exécute également `tests/python/test_auth_403_http.py` sur un serveur PHP natif et des données **temporaires**, sans SQL ; ce test est annoncé ignoré si PHP est absent. Ces tests ne remplacent pas une vérification dans un navigateur réel.

Articulation avec les suites de tests manuels (`docs/13-*`)

Même avec **15 jobs CI automatisés**, une campagne humaine reste indispensable sur ordinateur pour valider :

1. les parcours métier complets, la navigation au clavier et les confirmations explicites en deux étapes (avec et sans JavaScript) dans un navigateur réel ;
2. l'expérience réelle de la boîte de dialogue navigateur de permission de géolocalisation sur `/preferences` et la qualité linguistique perçue des pages traduites en anglais ;
3. l'hébergement cible **LineServ** (sous-dossier éventuel, séparation des fichiers privés, absence de scripts de développement exposés) et la recette manuelle sur la base réellement utilisée par l'application de production, pendant une fenêtre pré-démo autorisée ; cette même base doit être restaurée à l'état de démonstration avant la démo. `minishop_test` reste réservée aux tests SQL automatisés de GitHub CI et n'est pas la cible manuelle.

Périmètre volontairement exclu du manuel : les tests responsive multi-largeurs (`320/375/768/1440 px`, cartes mobiles `data-label`, portrait/paysage téléphone) ont été retirés des suites manuelles car trop longs à exécuter sur l'ensemble des vues ; ils reposent désormais exclusivement sur les suites automatisées (`tests/php/responsive_contract.php` et `tests/node/responsive_navigateur.mjs`). L'absence de validation humaine exhaustive sur téléphones physiques est assumée et documentée.

L'ensemble des vérifications humaines retenues est formalisé dans [`docs/13-fiches-recette-uc.md`](13-fiches-recette-uc.md) (**164 scénarios** sur `UC-01` à `UC-14` et `TCX-01` à `TCX-24`, plus les **36 couples** de la matrice `RB-11`) et son journal [`docs/13-suivi-recette-uc.csv`](13-suivi-recette-uc.csv) (**200 lignes** initialisées à `NON JOUÉ`). L'intégrité structurelle et la synchronisation de ces deux fichiers avec le code (`Triggers::TRANSITIONS`, routes, fichiers cités et absence de retour des scénarios responsive manuels) sont vérifiées automatiquement par `python3 scripts/verifier_fiches_uc.py` dans les orchestrateurs de tests et en CI.